Riphah International University
Gulberg Green

RIPHAH
INTERNATIONAL
UNIVERSITY

ASSIGNMENT NO : 02

Name : Hina Bibi

Sap ID : 69135

Department : BSSE-3

Course : Data Structure

Submitted to : Nimra Waqar

Date : 24/02/2026

DATE: __/__/____ DAY: __/__/____

Question No 1:

A singly linked list contains nodes where each node has two parts: Data and Next.
Explain the following elements of a singly linked list:

Node:

A node is the basic building block of a linked list. It is an object that holds two things: the data and a reference (pointer) to the next node.

data	next

Data field:

The data field stores the actual value or information that the node carries (eg: integar, string or any data type

42	next

data field

Next pointer:

The next pointer is a reference that points to the next node in the sequence, linking nodes together. The last node's next pointer is usually NULL (indicating the end of the list).

DATE: __/__/____ DAY: __/__/____

| data | next * | → | data | next * | → | data | NULL |

Head pointer:

The head pointer is a special reference that points to the first node of the list, giving access to the entire list. If the list is empty, the head is NULL.

head → | data | next | → | data | next | → | data | NULL |

head → 10 * → 20 * → 30 * → NULL

Part A: Insertion Operations

Write pseudocode and dry run:

1) Insertion at the Beginning:

Insert 10 into list: 20 → 30 → 40

head → 20 → 30 → 40 → NULL

Pseudocode:

```
Node *newNode;
newNode = new Node;
newNode -> data = 10;

newNode -> next = head;
head = newNode;
```

Dry Run:

Initial List

head → 20 → 30 → 40 → NULL

DATE: ___/___/___ DAY: ___/___/___

Step 1:

Create new node

newNode → 10 → NULL

Step 2:

newNode → next = head

List becomes:

10 → 20 → 30 → 40

Step 3:

head = newNode

Final List:

head → 10 → 20 → 30 → 40 → NULL

2) Insertion at End:

Insert 50 into list 10 → 20 → 30

head → 10 → 20 → 30 → NULL

Pseudocode:

```
Node *ptr;
Node *newNode;

newNode = new Node;
newNode -> data = 50;
newNode -> next = NULL;

ptr = head;

while (ptr ->next != NULL) {
```

DATE: __/__/____ DAY: __/__/____

```
    ptr = ptr -> next;
}
ptr -> next = newNode;
```

Dry Run:

Initial list

head -> 10 -> 20 -> 30 -> NULL

Pointer Movement:

ptr -> 10
ptr -> 20
ptr -> 30

Link new node:

30 -> 50 -> NULL

Final list:

head -> 10 -> 20 -> 30 -> 50 -> NULL

3) Insertion at a Specific Position:

Insert 25 at position 3 into list: 10 -> 20 -> 30 -> 40

head -> 10 -> 20 -> 30 -> 40 -> NULL

Pseudo code:

```
Node *ptr;
Node *newNode;

newNode = new Node;
```

DATE: ___/___/______ DAY: ___/___/______

```
newNode -> data = 25;

ptr = head;

for (int i = 1 ; i < position - 1 ; i++ ){
    ptr = ptr -> next;
}
    newNode -> next = ptr -> next ;
    ptr -> next = newNode;
```

Dry Run:

Initial:

head -> 10 -> 20 -> 30 -> 40 -> NULL

Move ptr:

ptr -> 10

ptr -> 20

Linking:

newNode -> next = 30

20 -> next = 25

Final list:

head -> 10 -> 20 -> 25 -> 30 -> 40 -> NULL

DATE: ___/___/_____ DAY: ___/___/_____

Part B: Deletion Operations

Write pseudocode and dry run:

4) Deletion from Beginning:

Delete first node from list: 10 -> 20 -> 30

head -> 10 -> 20 -> 30 -> NULL

Pseudocode:

```
Node *ptr;
ptr = head;

head = head -> next;
delete ptr;
```

Dry Run:

Initial:

head -> 10 -> 20 -> 30

Step 1:

ptr = 10

Step 2:

head = 20

Step 3:

delete ptr = 10.

Final List:

head -> 20 -> 30 -> NULL

DATE: __/__/__ DAY: __/__/__

5) Deletion from End:

Delete last node from list : 10→20→30→40

head → 10 → 20 → 30 → 40 → NULL

Pseudocode:

```
Node *ptr;
Node *predPtr;

ptr = head;
predPtr = NULL;

while (ptr->next != NULL){
    predPtr = ptr;
    ptr = ptr->next;
}
predPtr->next = NULL;
delete ptr;
```

Dry Run:

Initial:

head → 10 → 20 → 30 → 40 → NULL

Pointer movement:

predPtr = 10 , ptr = 20

predPtr = 20 , ptr = 30

predPtr = 30 , ptr = 40

Delete:

delete ptr = 10

DATE: ___/___/_____ DAY: ___/___/_____

Final list :

head → 10 → 20 → 30 → NULL

6) Delete at Specific Position :

Delete node at position 2 from list : 10 → 20 → 30 → 40

head → 10 → 20 → 30 → 40 → NULL

Pseudocode :

```
Node *ptr;
Node *predPtr;

ptr = head;
predPtr = NULL;

for(int i = 1; i < position; i++){
    predPtr = ptr;
    ptr = ptr->next;
}
    predPtr -> next = ptr->next;
    delete ptr;
```

Dry Run :

Initial :

head → 10 → 20 → 30 → 40 → NULL

ptr = 10

predPtr = NULL

DATE: __/__/__ DAY: __/__/__

Loop Runs Once:

predPtr = 10

ptr = 20

Change Link:

10 -> next = 30

List becomes:

10 -> 30 -> 40

Delete Node:

delete 20.

Final List:

head -> 10 -> 30 -> 40 -> NULL